

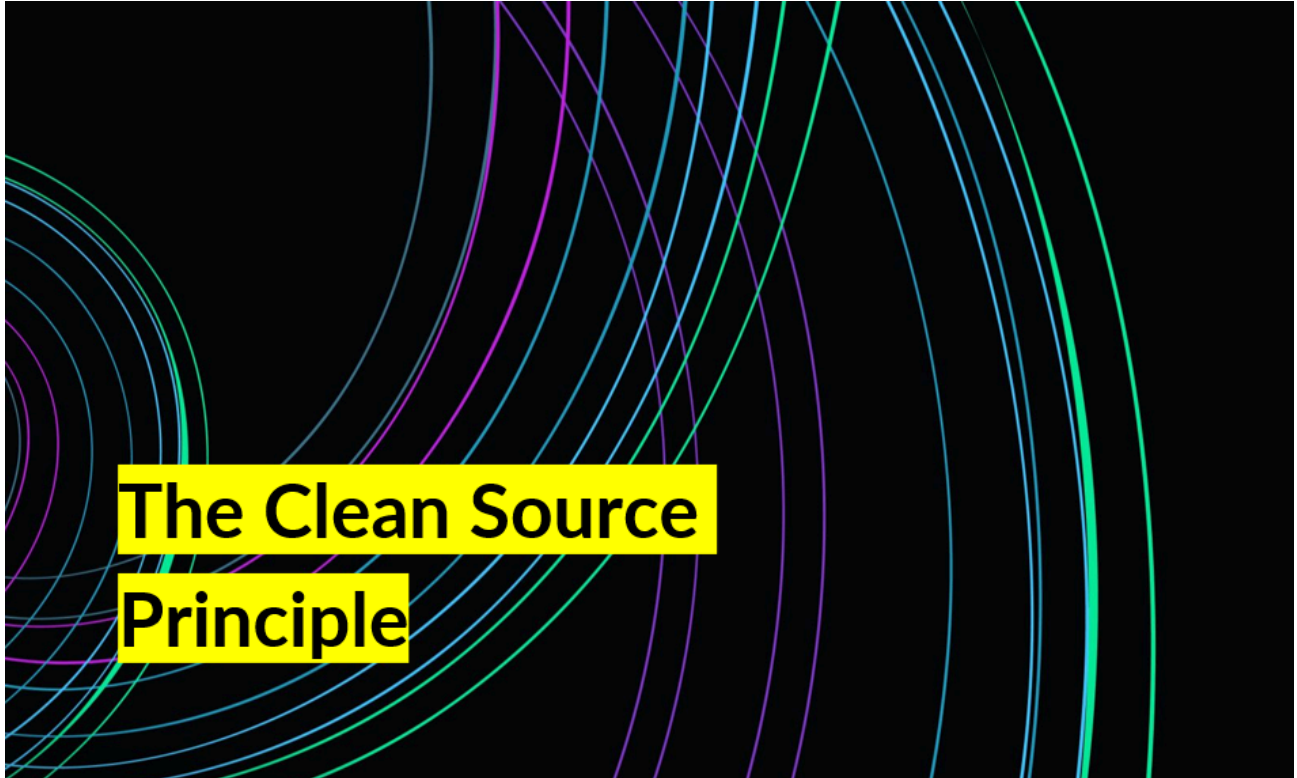
The Clean Source Principle – Automate the generation of up-to-date Windows ISO files. – Michael Waterman



michaelwaterman.nl/2023/06/20/the-clean-source-principle-securely-creating-up-to-date-iso-files

Michael Waterman

June 20, 2023



In today's interconnected world, cybersecurity is not just a technical consideration but a vital component of any organization's strategy. One essential principle to uphold in maintaining a secure environment is the 'clean source' principle. This principle dictates that we should trust only those software and systems that we can verify come from a trusted, clean source.

To support this idea, this blog post will demonstrate how to create a secure predictable environment for the creation of up to date of ISO files. We will be using Microsoft Deployment Toolkit (MDT) and PowerShell, both widely trusted and accessible tools.

The importance of clean source

When dealing with software, the clean source principle is of utmost importance for cybersecurity. It ensures that the software in use has not been tampered with and is free from malicious elements that could harm the system or the user. Essentially, by maintaining a clean source, we are guaranteeing the authenticity, integrity, and safety of our software.

The clean source principle serves as the first line of defense in securing our digital assets. In an era where threats are increasingly sophisticated, and the impacts of security breaches can be devastating, adherence to this principle is more important than ever.

This is especially true in enterprise environments where the scale and complexity of systems introduce countless potential points of entry for cyber threats. By ensuring that all software deployed comes from a clean source, we significantly reduce the attack surface, minimizing the potential for unauthorized access or damage.

Moreover, a clean source not only protects against malicious intent but also against inadvertent errors or vulnerabilities that may be present in unofficial or unverified copies of software. Hence, adhering to the clean source principle is as much about maintaining operational stability and performance as it is about cybersecurity.

In the following sections, we will discuss how to practically implement the clean source principle through the creation of up-to-date ISO files using MDT and PowerShell.

Overview of required tools

This blog post will demonstrate how to create a secure environment for the creation of up-to-date ISO files. We will be using Microsoft Deployment Toolkit (MDT), ADK, PowerShell, Visual Studio Code and Hyper-V, all of which are widely trusted and accessible tools from Microsoft.

MDT is a robust tool that allows for the efficient, automated deployment of Windows and related software. It helps ensure we can trust the source of the software we're deploying since it relies on Microsoft's own trusted sources.

PowerShell, on the other hand, is a task automation and configuration management framework. Its command-line shell and scripting language make it especially useful for automating complex, repetitive tasks, like the creation of ISO files.

Hyper-V is Microsoft's hardware virtualization product, enabling users to create and run virtual machines. These virtual environments provide an additional layer of security by isolating the software and allowing for secure testing and deployment. Additional software will be addressed later in this blog.

In the following sections, we will guide you on leveraging these tools to maintain a 'clean source' throughout the ISO creation process.

Building a predictable ISO creation environment, getting started

When it comes to establishing an environment for securely creating up-to-date ISO files, it's crucial to extend the 'clean source' principle to the foundation of our systems – the Windows environment. This base must be as secure and trusted as the software we're planning to deploy. That means our operating system should be installed using trusted media, sourced directly from Microsoft. This practice not only guarantees that we're using a legitimate, unaltered version of the operating system, but it also equips us with the latest security updates and patches right off the bat. By adopting this method, we strengthen the security posture of our entire environment, thereby increasing the reliability and currency of our ISO files and, consequently, the software we deploy from them. In the following sections, we will delve into the steps to construct this secure environment for producing up-to-date ISO files using trusted, clean sources.

One key element of establishing a secure environment for creating up-to-date ISO files is using a trusted virtualization platform. For this, we'll leverage Microsoft's own Hyper-V. It allows us to create a shielded environment where we can safely generate and work with our ISO files. Regardless of the Windows version, using Hyper-V gives us an isolated, secure place to build and test our deployments.

For this tutorial, we're going to use Windows 11, but the process will be similar for other versions of Windows.

To begin with, we need to install the Hyper-V platform on our Windows 11 system. This can be accomplished easily using PowerShell with Administrator rights. Open a new PowerShell window as an administrator and run the following command:

PowerShell

Windows Client:

```
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V -All
```

Windows Server:

```
Install-WindowsFeature -Name Hyper-V -IncludeManagementTools
```

This command enables the Hyper-V feature in Windows, installing all necessary components.

Note! If you would like to test these features within a VM, you can enable Hyper-V Para-Virtualization, use the following command while the machine is turned off.

PowerShell

```
Set-VMProcessor -VMName <VMName> -ExposeVirtualizationExtensions $true
```

Working in a shielded environment like Hyper-V offers several key security enhancements. For one, it adds an additional layer of abstraction between the host system and the

software being developed, thereby isolating potential threats. Also, it enables us to test our ISO files in a secure environment before deploying them.

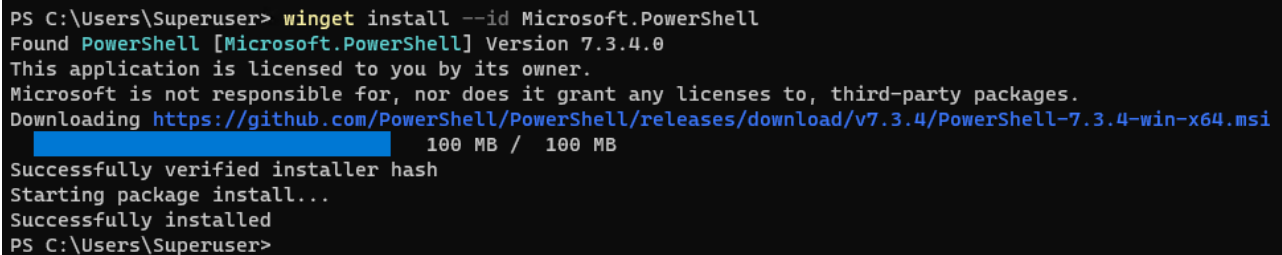
Setting up the required software

Before diving into the installation and configuration of the tools, we need to ensure we have all the necessary software installed on our system. For our purposes, these include PowerShell, Visual Studio Code, Windows Assessment and Deployment Kit (ADK), Windows Assessment and Deployment Kit Windows Preinstallation Environment (WinPE), and Microsoft Deployment Toolkit (MDT). Open an elevated Windows Terminal shell for entering the commands displayed below.

PowerShell

We'll start with PowerShell, which is a cross-platform task automation solution made up of a command-line shell, a scripting language, and a configuration management framework. We'll be using PowerShell version 7, the latest version, due to its speed, security, and compatibility improvements.

```
winget install --id Microsoft.PowerShell
```

A screenshot of a Windows PowerShell terminal window showing the command 'winget install --id Microsoft.PowerShell' and its output. The output indicates that PowerShell version 7.3.4.0 was found, the application is licensed to the user, and it was successfully installed. A progress bar shows the download of the installer file from GitHub.

```
PS C:\Users\Superuser> winget install --id Microsoft.PowerShell
Found PowerShell [Microsoft.PowerShell] Version 7.3.4.0
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://github.com/PowerShell/PowerShell/releases/download/v7.3.4/PowerShell-7.3.4-win-x64.msi
100 MB / 100 MB
Successfully verified installer hash
Starting package install...
Successfully installed
PS C:\Users\Superuser>
```

Note! Since winget is not available on the server version of Windows, the package needs to be [downloaded](#) and installed manually.

Visual Studio Code

Visual Studio Code, often referred to as VS Code, is a lightweight but powerful source code editor developed by Microsoft. It runs on your desktop and is available for Windows, macOS, and Linux.

VS Code comes with built-in support for a variety of languages and is highly customizable, so you can add extensions and themes to make it suit your preferences or needs. It has features like IntelliSense code completion, code refactoring, and debugging that enhance the coding experience, making it easier and more efficient.

```
winget install --id Microsoft.VisualStudioCode
```

For our purpose of creating a secure environment for generating up-to-date ISO files, using VS Code will be extremely beneficial for managing and editing our scripts.

Note! Since winget is not available on the server version of Windows, the package needs to be [downloaded](#) and installed manually.

Windows Assessment and Deployment Kit (ADK)

Next, we need the Windows ADK, a collection of tools that you can use to customize, assess, and deploy Windows operating systems to new computers. It's a key component in creating our secure environment for generating up-to-date ISO files.

```
winget install --id Microsoft.WindowsADK
```

Note! Since winget is not available on the server version of Windows, the package needs to be [downloaded](#) and installed manually.

Windows Assessment and Deployment Kit Windows Preinstallation Environment (WinPE)

WinPE, part of Windows ADK, is a small operating system used to install, deploy, and repair Windows. It's an essential tool in our toolkit, particularly when we're dealing with system images and deployments.

```
winget install --id Microsoft.ADKPEAddon
```

Note! Since winget is not available on the server version of Windows, the package needs to be [downloaded](#) and installed manually.

Microsoft Deployment Toolkit (MDT)

Finally, we need to install MDT, a solution accelerator for operating system and application deployment. MDT offers us a unified collection of tools, processes, and guidance for automating desktop and server deployments.

```
winget install --id Microsoft.DeploymentToolkit
```

With all these tools installed, we're ready to move forward to the installation and configuration of the automation tools.

Note! Since winget is not available on the server version of Windows, the package needs to be [downloaded](#) and installed manually.

Source media

One of the key steps in securely creating up-to-date ISO files is obtaining the source media. For the purposes of this blog post, we'll be using the evaluation version of Windows Server 2022, which is a fully-featured version of Microsoft's enterprise-level server operating system. Remember, the important part here is to always get your source media from trusted sources. In this case, we're getting it directly from Microsoft, which ensures

that it hasn't been tampered with and that it is in its original, secure state. Also, it's worth mentioning that while we're using the Windows Server 2022 evaluation version here, you could use any supported version following the same steps.

To streamline this process, we'll be using PowerShell to download the ISO file for Windows Server 2022. This practice not only saves time and guarantees the accuracy of our actions, but it also reinforces our commitment to the clean source principle by ensuring our source media is directly downloaded from a reputable source, untouched by potential threat actors.

In the following steps, I'll demonstrate how to get the ISO file using PowerShell. Open up an elevated Terminal shell and use the following command to obtain the ISO file for Windows Server 2022. Please note that it's a large file and depending on your bandwidth it could take a while to download.

PowerShell

```
Invoke-WebRequest -Uri 'https://go.microsoft.com/fwlink/p/?  
LinkID=2195280&clcid=0x409&culture=en-us&country=US' -outfile WinServer2022Eval.iso
```

To obtain the part after the Uri, I've found [this script](#) very helpful.

Adhering to the principle of least privilege: Creating a dedicated MDT user

As we continue our journey to securely create up-to-date ISO files, it's vital to consider the cybersecurity principle known as the 'Principle of Least Privilege' (PoLP). This principle suggests that in a particular system, a user should be given the minimum levels of access necessary to complete their tasks. This method significantly reduces the potential damage from accidents or malicious actions.

In the context of our deployment environment, this principle guides us to create a dedicated user for the Microsoft Deployment Toolkit (MDT). This MDT user should be granted only enough permissions to read the deployment share and upload to the capture folder. By doing this, we minimize the potential attack surface and ensure that even if this user's credentials were compromised, the impact would be as limited as possible.

Creating a dedicated MDT user not only aligns with the 'clean source' principle but also upholds the 'Principle of Least Privilege', ensuring a more secure, reliable, and resilient deployment environment. Use the Following script to create the user. Execute it from an elevated Visual Studio Code.

New-MDTUser.ps1

```

$DeploymentUserName = "MDTUser"
$DeploymentUserPassword = [System.Net.NetworkCredential]::new("", (Read-Host -
AsSecureString -Prompt "Enter Password") ).Password

# We need the following in PS7 as it does not support 32bit Modules in 64bit.
import-module microsoft.powershell.localaccounts -UseWindowsPowerShell

if (!(Get-LocalUser -Name $DeploymentUserName -ErrorAction SilentlyContinue)){
    New-LocalUser -Name $DeploymentUserName `
        -Password ($DeploymentUserPassword | convertto-securestring -asplaintext -
Force) `
        -AccountNeverExpires `
        -Description $DeploymentUserName `
        -FullName "MDT Deployment User" `
        -PasswordNeverExpires `
        -UserMayNotChangePassword `
        | Out-Null
}

If (!(Get-LocalGroupMember -Group "Users" -Member $DeploymentUserName -ErrorAction
SilentlyContinue)){
    Add-LocalGroupMember -Group Users -Member $DeploymentUserName
}

```

Establishing a file share for the updated ISO files

After creating our dedicated MDT user, the next step is to establish a secure location where our updated ISO files will be published. For this, we'll create a file share using PowerShell. This approach not only automates the process but also ensures that the actions are accurate and replicable, upholding our commitment to the 'clean source' principle.

Creating a file share involves several steps: defining the location of the share, setting up the share itself, and then assigning the appropriate permissions. Each of these steps is crucial for maintaining a secure environment.

Selecting the Location

Firstly, we need to decide where our file share will be located and what it will be named. This could be any directory on your machine that can be accessed by the MDT user. In this example we'll put the share of the D: drive, create a folder named "share" and share it on the network under the name "share". Variables define this structure.

Creating the file share

Once the location and name has been determined, we can create the file share using PowerShell. Finally, we need to assign permissions to the file share. Remember, our dedicated MDT user should have Read/Write to the share. Once the file share is

established and the necessary permissions are set, we're ready to proceed with the next steps. The entire code is listed below.

New-ISOShare.ps1

```
$CurrentDriveLetter = "D:"
$RootShareFolderName = "Share"
$RootShareName = "Share"
$DeploymentUserName = "MDTUser"

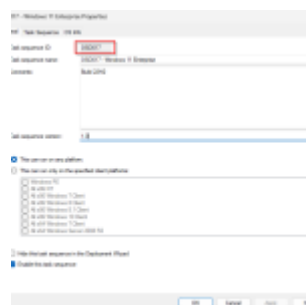
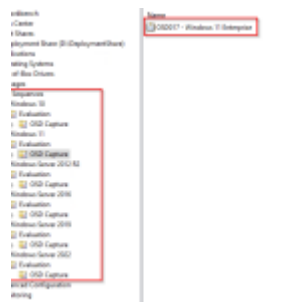
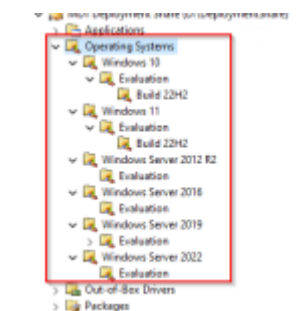
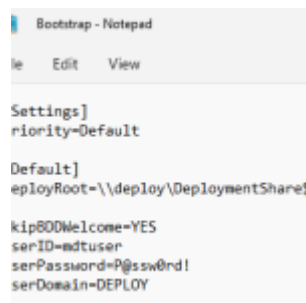
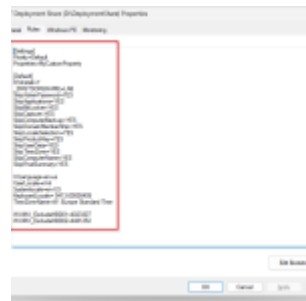
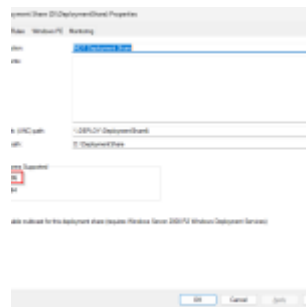
If (!(Test-Path (Join-Path -Path $CurrentDriveLetter -ChildPath
$RootShareFolderName) )){
    New-Item -Path (Join-Path -Path $CurrentDriveLetter -ChildPath
$RootShareFolderName) -ItemType Directory
}

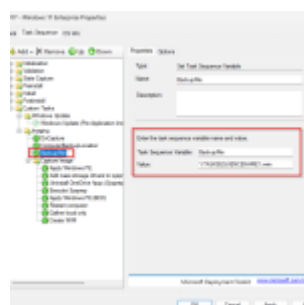
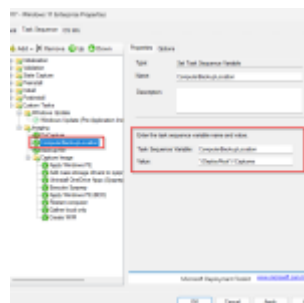
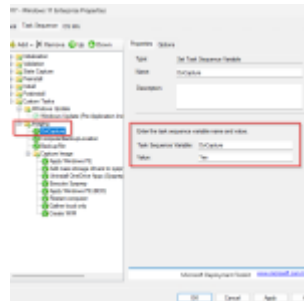
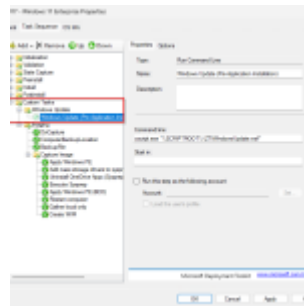
if (!(Get-SmbShare -Name $RootShareName -ErrorAction SilentlyContinue) ){
    New-SmbShare -Name $RootShareName `
        -Path (Join-Path -Path $CurrentDriveLetter -ChildPath $RootShareFolderName)
    ,
        -CachingMode None `
        -FullAccess "Administrators" `
        -ChangeAccess $DeploymentUserName
}
```

Working with Microsoft Deployment Toolkit (MDT)

As we delve deeper into our journey of securely creating up-to-date ISO files, we cannot overlook the significant role played by Microsoft Deployment Toolkit (MDT). MDT offers us a unified collection of tools, processes, and guidance for automating desktop and server deployments, which is crucial for our endeavor. Setting up MDT can be quite involved, and there are numerous guides available that provide step-by-step instructions for setting up an MDT environment. Our aim here is not to reproduce those guides. Instead, we want to focus on how we can leverage MDT in our specific context to uphold the 'clean source' principle and enhance the security of our deployment environment.

I will share my configuration files and screenshots to demonstrate how I've set up MDT for this purpose. We will look into some crucial aspects, such as creating the image and modifying the task sequence to fit our needs. If an item is not listed below that means that I've used the default configuration.



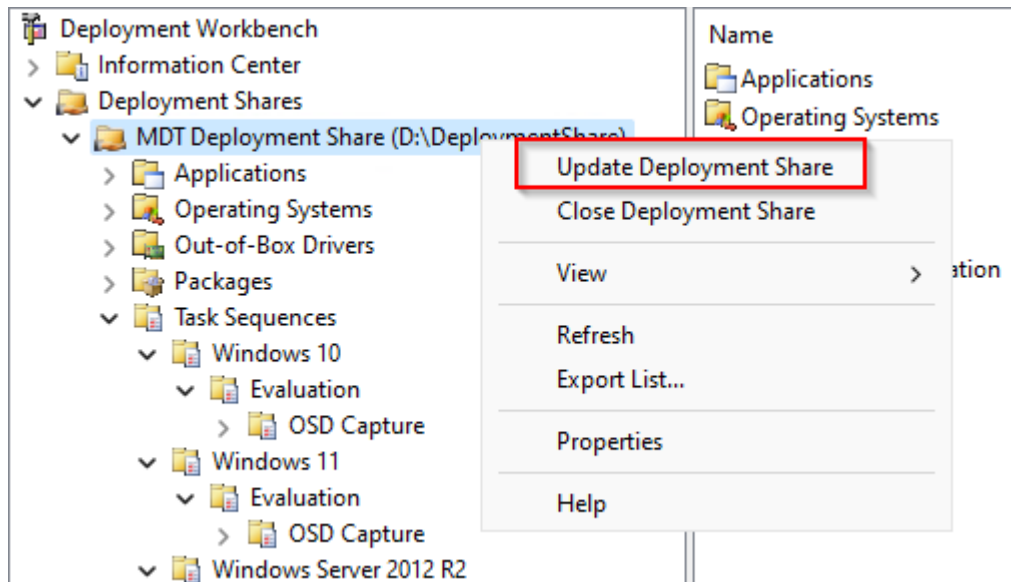


Simply put, create a new deployment share, change the default settings to match those I've displayed above (or from the download folder at the end of this blog).

Updating the MDT media share and addressing Windows 11 ADK issues

As part of maintaining a secure, up-to-date environment for ISO generation, it's essential that we periodically update the MDT media share. This process ensures that we always have the latest boot media and any changes or additions we've made to the deployment share are incorporated into the boot media.

Updating the MDT media share can be done by right clicking the deployment share and select *"Update Deployment Share"* – *"Completely regenerate the boot images"*.



However, if you've upgraded to the new version of the Windows 11 ADK, you might have encountered an error with the HTA Deployment Wizard. This issue is known, and until a permanent solution is provided, we have a workaround that we can apply to ensure the smooth running of our process.

The workaround involves copying the content of the XML file below to the "Unattend_PE_x64.xml" file, located in "C:\Program Files\Microsoft Deployment Toolkit\Templates". Make sure that you save a copy of the original file first!

Unattend_PE_x64.xml

```
<unattend xmlns="urn:schemas-microsoft-com:unattend">
  <settings pass="windowsPE">
    <component name="Microsoft-Windows-Setup" processorArchitecture="amd64"
      publicKeyToken="31bf3856ad364e35" language="neutral" versionScope="nonSxS"
      xmlns:wcm="http://schemas.microsoft.com/WMIConfig/2002/State">
      <Display>
        <ColorDepth>32</ColorDepth>
        <HorizontalResolution>1024</HorizontalResolution>
        <RefreshRate>60</RefreshRate>
        <VerticalResolution>768</VerticalResolution>
      </Display>
      <RunSynchronous>
        <RunSynchronousCommand wcm:action="add">
          <Description>Lite Touch PE</Description>
          <Order>1</Order>
          <Path>reg.exe add "HKLM\Software\Microsoft\Internet
Explorer\Main" /t REG_DWORD /v JscriptReplacement /d 0 /f</Path>
        </RunSynchronousCommand>
        <RunSynchronousCommand wcm:action="add">
          <Description>Lite Touch PE</Description>
          <Order>2</Order>
          <Path>wscript.exe X:\Deploy\Scripts\LiteTouch.wsf</Path>
        </RunSynchronousCommand>
      </RunSynchronous>
    </component>
  </settings>
</unattend>
```

```
        </component>
    </settings>
</unattend>
```

After applying the workaround, it's important to update the deployment share again. This ensures that the workaround is incorporated into the boot media.

Creating an updated MDT LiteTouch boot media

Now that we have configured our MDT, our next step is to create an updated version of the MDT LiteTouch boot media. This is crucial for our automated process. The standard LiteTouch ISO file displays a prompt at boot-up that says "press any key to boot from CD or DVD". This feature is great for manual deployments as it prevents unintentional booting from the media. However, it presents a hurdle for our automated process as it requires manual intervention.

To resolve this, I have created a PowerShell script that takes the original LiteTouch ISO file and creates an updated version that bypasses the "press any key" prompt. This allows us to maintain the automation of our process, reducing the need for manual intervention and the possibility of human error.

The script works by modifying the boot sector of the original LiteTouch ISO file. This effectively disables the "press any key" prompt, allowing the system to boot directly from the media.

New-LiteTouchPE_x64_NoPrompt.ps1

```
# Repack ISO to disable 'press any key' function

# Set variables
$LiteToucheIsoPath = "D:\DeploymentShare\Boot\LiteTouchPE_x64.iso"
$DestinationFolder = "D:\Temp"
$ISODestinationName = "LiteTouchPE_x64_NoPrompt.iso"

# Searching for the OSCDIMG tool on the C:\ drive
$Oscding = Get-ChildItem -Path "c:\\" `
    -Filter "OSCDIMG.EXE" `
    -Recurse `
    -ErrorAction SilentlyContinue `
    -Force | ForEach-Object { $_.FullName } | Select-String -
Pattern "amd64"

# Obtain the directory
$OscdingFolder = Split-Path $Oscding -Parent
If ($Oscding -eq "") {
    Write-Verbose "Warning! Not found..."
}
Else
{
```

```

        Write-Verbose "OK..."
    }

# Set the path to the boot files located in the OSCDIMG directory
$EtfBoot = "$OscdImgFolder\etfsboot.com"
$EfiSys = "$OscdImgFolder\efisys_noprompt.bin"

# Mount Original LiteToche ISO file.
Mount-DiskImage -ImagePath $LiteToucheIsoPath -StorageType ISO

# Get drive letter for the mounted
$DriveLetter = (Get-DiskImage -ImagePath $LiteToucheIsoPath | Get-Volume).DriveLetter + ":"

# Copy ISO files to the temp folder
$ScratchFolder = Join-Path -Path $env:temp -ChildPath "iso"
If (!(Test-Path ($ScratchFolder) )) {
    New-Item -Type Directory -Path $ScratchFolder
}
Copy-Item $DriveLetter\* $ScratchFolder -Force -Recurse

# Unmount LiteTouch ISO file.
Dismount-DiskImage -ImagePath $LiteToucheIsoPath

# Remove read-only attributes from all files
Get-ChildItem $ScratchFolder -Recurse | ForEach-Object{ if (!$_.psiscontainer) {
    $_.isreadonly = $false}}

# Construct the bootdata
$BootData = '2#p0,e,b"{0}"#pEF,e,b"{1}"' -f $EtfBoot, $EfiSys

# Create the destination folder if it does not exist
If (!(Test-Path ($DestinationFolder) )) {
    New-Item -Type Directory -Path $DestinationFolder
}
# Remove any previous iso file with the same name
If ((Test-Path (Join-Path -Path $DestinationFolder -ChildPath $ISODestinationName)
)) {
    Remove-Item (Join-Path -Path $DestinationFolder -ChildPath $ISODestinationName)
-Force
}

# Start building the new ISO
Start-Process $oscdimg -args @("-bootdata:$BootData", '-m', '-o', '-u2', '-udfver102', $ScratchFolder, (Join-Path -Path $DestinationFolder -ChildPath $ISODestinationName)) -wait -nonewwindow

# Cleanup the temp files
If ((Test-Path ($ScratchFolder) )) {
    Remove-Item -LiteralPath $ScratchFolder -Recurse
}

```

With this updated LiteTouch boot media, we've removed one more manual step from our process, bringing us one step closer to our goal of a secure, automated environment for creating up-to-date ISO files.

Tweaking NTFS Permissions: Empowering Our MDTUser

When working with a default deployment share and using a dedicated, limited user account (in our case, the MDTUser), we may encounter permission issues. By default, the MDTUser might not have the necessary rights to read data from the deployment share or write WIM files to the capture folder due to NTFS permissions. To overcome this, we can use PowerShell to adjust the permissions accordingly.

Here is how we ensure our MDTUser has the appropriate rights to carry out its duties. First, we need to adjust the permissions on the deployment share. With a PowerShell script, we can grant the MDTUser 'Read' permissions to the contents of the deployment share:

Next, we need to adjust the permissions on the capture folder. By modifying the NTFS permissions, we allow the MDTUser to write WIM files to the capture directory. This is the script I use:

Set-DeploymentACL.ps1

```
$MDTPath = "D:\DeploymentShare"
$identity = 'MDTUser'

# Set the DeploymentShare Generic NTFS Rights
$ACL = Get-Acl -Path $MDTPath

# Clean the ACL
$acl.Access | ForEach-Object{$acl.RemoveAccessRule($_)}

# Add SYSTEM full control on the deployment root
$rule = New-Object System.Security.AccessControl.FileSystemAccessRule("NT
AUTHORITY\SYSTEM", "FullControl", "ContainerInherit, ObjectInherit", "None",
"Allow")
$Acl.AddAccessRule($rule)

# Add the administrators group full control on the deployment root
$rule = New-Object
System.Security.AccessControl.FileSystemAccessRule("BUILTIN\Administrators",
"FullControl", "ContainerInherit, ObjectInherit", "None", "Allow")
$Acl.AddAccessRule($rule)

# Add the current user full control on the deployment root
$rule = New-Object
System.Security.AccessControl.FileSystemAccessRule("$(($Env:UserName)",
"FullControl", "ContainerInherit, ObjectInherit", "None", "Allow")
```

```

$Acl.AddAccessRule($rule)

# Add the MDT User Read and execute on the deployment root
$rule = New-Object System.Security.AccessControl.FileSystemAccessRule($identity,
"ReadAndExecute", "ContainerInherit, ObjectInherit", "None", "Allow")
$Acl.AddAccessRule($rule)

Set-Acl -Path $MDTPath -AclObject $Acl

# Reset the ACL
$ACL = $null

# Obtain the ACL from the capture directory
$ACL = Get-Acl -Path $(Join-Path -Path $MDTPath -ChildPath "Captures")

# Add new rule
$rule = New-Object System.Security.AccessControl.FileSystemAccessRule($identity,
"Write, Synchronize", "ContainerInherit, ObjectInherit", "None", "Allow")
$Acl.AddAccessRule($rule)

# Set the new ACL
Set-Acl -Path $(Join-Path -Path $MDTPath -ChildPath "Captures") -AclObject $Acl

# Add the MDTUser to the MDT Share
Grant-SmbShareAccess -Name $((Get-SmbShare | Where-Object Path -eq
"D:\DeploymentShare").Name) -AccountName $identity -AccessRight Change -Force

```

By using this method, we ensure that our MDTUser has just enough permissions to perform its tasks, adhering to the principle of least privilege, and further ensuring the security and integrity of our deployment process.

Creating virtual machines for installations

With the updated MDT LiteTouch boot media in hand, our next step in the process is to create several virtual machines that will serve as our base installations. Using virtual machines provides flexibility and scalability in our process, and makes reverting to a previous state easy, thereby increasing the reliability of our approach.

To streamline this task, I have developed a PowerShell script that not only creates these virtual machines but also prepares them for the next steps. Post-creation, a snapshot is taken to save the state of the virtual machines. This snapshot acts as a point of reference to which the virtual machines can easily be reverted if needed, enhancing our control over the process.

After creating the snapshot, the script automatically attaches the updated LiteTouch ISO file to each virtual machine. This means that upon booting, each virtual machine will start up from the LiteTouch boot media, bypassing the “press any key” prompt and leading directly to the MDT deployment wizard. Here’s how you can use this script:

New-DeploymentVMs.ps1

```
# Define the variables
# Create an array of Virtual Machines.
$VMNames = @("Windows Server 2022")

# Where will the Virtual Disks be stored.
$VMDiskPath = "D:\Hyper-v\Virtual Hard Disks"

# Which VMSwitch will be used (use get-VMSwitch).
$VMSwitch = "Default Switch"

# Path to the altered LiteTouche ISO file.
$BootISOPath = "D:\Temp\LiteTouchPE_x64_NoPrompt.iso"

# Check / Create VMSwitch "Default Switch"
if (!(Get-VMSwitch | Where-Object Name -eq "Default Switch")){
    New-VMSwitch -name ("Default Switch") -NetAdapterName $((Get-NetAdapter |
select -First 1).name) -AllowManagementOs $true
}

ForEach($VMName in $VMNames){

    If(Test-Path (Join-Path -Path $VMDiskPath -ChildPath ($VMName + ".vhdx") )){

        Remove-Item -Path (Join-Path -Path $VMDiskPath -ChildPath ($VMName +
".vhdx")) -Force
    }

    if(Get-VM -Name $VMName -ErrorAction SilentlyContinue){
        Remove-VM -Name $VMName -Force
    }

    # Create the VM
    New-VM -Name $VMName -Generation 2 -SwitchName $VMSwitch -NewVHDPATH (Join-Path
-Path $VMDiskPath -ChildPath ($VMName + ".vhdx")) -NewVHDSIZE 127GB

    # Set the options
    Set-VMProcessor -VMName $VMName -Count 4
    Add-VMDvdDrive -VMName $VMName -Path $BootISOPath
    Set-VMFirmware -VMName $VMName -FirstBootDevice (Get-VMDvdDrive -VMName
$VMName)
    Set-VM -VMName $VMName -AutomaticCheckpointsEnabled $False

    # Create a snapshot
    Checkpoint-VM -Name $VMName -SnapshotName "Before OS Deployment"

    # Start the VM
    Start-VM -VMName $VMName
}
```


By using this script, we automate the creation and setup of our base installations, increasing the efficiency of our process and further solidifying our commitment to the 'clean source' principle.

Note! When booting this lab setup on a single Windows Server without support of DHCP on the network, the VM will not be able to obtain a DHCP address. This works differently on Windows clients where the "Default Switch" provides DHCP capabilities.

Automating ISO generation with a scheduled task

As we've progressed through the process of creating a secure environment for generating up-to-date ISO files, we've automated each step as much as possible. We're now at the point where we're ready to automate the actual generation of the ISO files themselves. For this, I've developed a PowerShell script that integrates with a scheduled task to automatically generate new ISO files.

The script works by reading a configuration XML file. This XML file contains all the necessary details for generating the updated ISO files. It uses the "Task Sequence Reference ID" as a marker to identify which WIM images to process into an ISO file.

By default, the scheduled task runs every 4 hours. During each run, it reads the configuration from the XML file, identifies any newly generated WIM images, and processes them into a new ISO file, making the entire process fully automated. This not only ensures the 'clean source' principle is upheld but also increases the efficiency and consistency of our process.

The source code for this script and the XML configuration file will be made available on my blog page as a downloadable file. But before we delve into the specifics of the script, let's first understand the different options in the script and the XML configuration file.

New-WindowsInstallationMedia.ps1

```
<#
```

```
.Synopsis
```

```
    Create a new bootable ISO Installation media with parameters derived from a XML configuration file.
```

```
.DESCRIPTION
```

```
    This script creates a new bootable ISO Installation media with parameters derived from a XML configuration file.
```

```
    This configuration file has all the settings that are used during the creation of the ISO. This script requires elevated privileges.
```

```
.Parameter XMLFile
```

```
    path to the XML the configuration file.
```

```
.Notes
    Version 1.0

    Date: 22-06-2022
#>

Param (
    [Parameter(Mandatory=$true)]
    [string]$XMLFile
)

# Initialize objects for security operations
$wid=[System.Security.Principal.WindowsIdentity]::GetCurrent()
$prp=new-object System.Security.Principal.WindowsPrincipal($wid)
$adm=[System.Security.Principal.WindowsBuiltInRole]::Administrator
$IsAdmin=$prp.IsInRole($adm)

# Halt if the script is not running as admin
if($IsAdmin -eq $false)
{
    Write-Verbose "Process is not running as admin."
    exit
}

# Event Logging

$source = "SecureDeployment"

if ([System.Diagnostics.EventLog]::SourceExists($source) -eq $false)
{
    [System.Diagnostics.EventLog]::CreateEventSource($source, "Application")
}

# Functions
function Test-FileLock {
    param (
        [parameter(Mandatory=$true)][string]$Path
    )

    $oFile = New-Object System.IO.FileInfo $Path

    if ((Test-Path -Path $Path) -eq $false) {
        return $false
    }

    try {
        $oStream = $oFile.Open([System.IO.FileMode]::Open,
[System.IO.FileAccess]::ReadWrite, [System.IO.FileShare]::None)

        if ($oStream) {
            $oStream.Close()
        }
    }
}
```

```

        return $false
    } catch {
        # file is locked by a process.
        return $true
    }
}

# Test the XML Data file location
If(Test-Path $XMLFile){
    [XML]$XML = Get-Content -Path $XMLFile
} Else {
    Write-EventLog -EventId 1 -LogName Application -Message "XML Data File not Found." -Source SecureDeployment -Category 0 -EntryType Error
    Write-Verbose "XML Data File not Found."
    Return
}

# Test the Image capture folder
If(Test-Path $XML.Configuration.ImageData.CaptureFolder){
    # Folder exists, continue
} Else {
    Write-EventLog -EventId 2 -LogName Application -Message "Image Capture folder not found." -Source SecureDeployment -Category 0 -EntryType Error
    Write-Verbose "Image Capture folder not found."
    Return
}

# Test the ISO destination folder
If(Test-Path $XML.Configuration.ImageData.DestinationFolder){
    # Folder exists, continue
} Else {
    Write-EventLog -EventId 3 -LogName Application -Message "ISO destination folder not found." -Source SecureDeployment -Category 0 -EntryType Error
    Write-Verbose "ISO destination folder not found."
    Return
}

# Test the ISO Creation Utility
If(Test-Path $XML.Configuration.ImageData.Oscdimg){
    # File exists, continue
} Else {
    Write-EventLog -EventId 4 -LogName Application -Message "ISO creation utility OSCDIMG could not be located." -Source SecureDeployment -Category 0 -EntryType Error
    Write-Verbose "ISO creation utility OSCDIMG could not be located."
    Return
}

# Test the ImageX Utility
If(Test-Path $XML.Configuration.ImageData.ImageX){
    # File exists, continue
}

```

```

} Else {
    Write-EventLog -EventId 5 -LogName Application -Message "ImageX utility could
not be located." -Source SecureDeployment -Category 0 -EntryType Error
    Write-Verbose "ImageX utility could not be located."
    Return
}

# Check the Compression type
$CompressionType = "None", "Max", "Fast", "Recovery"
if ( $($XML.Configuration.ImageData.Compression) -in $CompressionType)
{
    # Compression Type correctly set
} Else {
    Write-EventLog -EventId 6 -LogName Application -Message "Compression Type not
set correctly. Use one of these options: None, Fast, Max or Recovery." -Source
SecureDeployment -Category 0 -EntryType Error
    Write-Verbose "Compression Type not set correctly. Use one of these options:
None, Fast, Max or Recovery."
    Return
}

# Get the Content of the capture folder
$WimFiles = Get-ChildItem -Path $XML.Configuration.ImageData.CaptureFolder -Filter
"*.wim"

If($WimFiles){
    foreach($WIMFile in $WimFiles){

        Write-EventLog -EventId 7 -LogName Application -Message "New WIM file with
name $WIMFile detected." -Source SecureDeployment -Category 0 -EntryType
Information

        # Check OpLock
        If(Test-FileLock -Path $WIMFile.Fullname){
            Write-EventLog -EventId 8 -LogName Application -Message "file $WIMFile
is locked, will try at next run." -Source SecureDeployment -Category 0 -EntryType
Warning
            Return
        }

        # Get/Set the Staging Folder
        If(Test-Path $XML.Configuration.ImageData.StagingFolder){
            Remove-Item -Path $XML.Configuration.ImageData.StagingFolder -Recurse -
Force
            New-Item -Path $XML.Configuration.ImageData.StagingFolder -ItemType
Directory -Force | Out-Null
        } Else {
            New-Item -Path $XML.Configuration.ImageData.StagingFolder -ItemType
Directory -Force | Out-Null
        }

        # Get the MDT Generated Image Name

```

```

        $MDTImageName = Get-WindowsImage -ImagePath $WIMFile.Fullname -Index 1 |
Select-Object -ExpandProperty ImageName

        # Search for the Conversion Name
        $FileName = Select-Xml -Xml $XML -XPath
        "-//OperatingSystem[@key='$MDTImageName']" | Select-Object -ExpandProperty Node |
Select-Object -ExpandProperty Name

        If(!$FileName){
            Write-EventLog -EventId 9 -LogName Application -Message "Mapping
between source and target WIM file could not be made." -Source SecureDeployment -
Category 0 -EntryType Error
            Write-Verbose "Mapping between source and target WIM file could not be
made."
            Return
        }

        # Search for the Image Name
        $ImageName = Select-Xml -Xml $XML -XPath
        "-//OperatingSystem[@key='$MDTImageName']/ImageName" | Select-Object -ExpandProperty
Node | Select-Object -ExpandProperty "#text"

        # Get the Source Folder
        $SourceFolder = Select-Xml -Xml $XML -XPath
        "-//OperatingSystem[@key='$MDTImageName']/Source" | Select-Object -ExpandProperty
Node | Select-Object -ExpandProperty "#text"

        # Copy The Source Content to the Staging Folder
        If(Test-Path $SourceFolder){
            Copy-Item -Path (Join-Path -Path $SourceFolder -ChildPath "\*") -
Destination $XML.Configuration.ImageData.StagingFolder -Exclude
@('install.wim','*.clg') -Recurse -Force
        } Else {
            Write-EventLog -EventId 10 -LogName Application -Message "Source Folder
Could not be located." -Source SecureDeployment -Category 0 -EntryType Error
            Write-Verbose "Source Folder Could not be located."
            Return
        }

        # Copy the Captured WIM File to the Staging Sources folder
        Start-Process -FilePath $($XML.Configuration.ImageData.DISM) -ArgumentList
@("/export-image /sourceImageFile:""$($WIMFile.Fullname)""
/destinationImageFile:""$($Join-Path $XML.Configuration.ImageData.StagingFolder -
ChildPath "sources\install.wim)"" /SourceName:""$($MDTImageName)""
/Compress:""$($XML.Configuration.ImageData.Compression)"" ") -PassThru -Wait -
NoNewWindow | Out-Null

        # Construct the current date
        $GetDate = Get-Date
        $Today = [string]$(($GetDate.Day) + '-' + [string]$(($GetDate.Month) + '-' +
[string]$(($GetDate.Year)

```

```

# Update the image name
Start-Process -FilePath $($XML.Configuration.ImageData.ImageX) -
ArgumentList @("/info ""$(Join-Path $XML.Configuration.ImageData.StagingFolder -
ChildPath sources\install.wim)"" 1 ""$(($ImageName)"" ""Image Creation Date:
$(($Today)"" "" ) -PassThru -Wait -NoNewWindow | Out-Null

# Create the hash file
"File created on: $($today)" | Out-File $(Join-Path
$XML.Configuration.ImageData.StagingFolder -ChildPath sources\install-source.hash)
-Force -Encoding unicode
Get-FileHash -Path $(Join-Path $XML.Configuration.ImageData.StagingFolder -
ChildPath sources\install.wim) -Algorithm SHA256 | Select-Object Algorithm, Hash |
Out-File $(Join-Path $XML.Configuration.ImageData.StagingFolder -ChildPath
sources\install-source.hash) -Append

# Create the ISO File Name
$ISOFile = (Join-Path $XML.Configuration.ImageData.DestinationFolder -
ChildPath ($FileName + " - " + $Today + ".iso" ) )

# Remove Previous ISO File
if(Test-Path $ISOFile){
    Remove-Item $ISOFile -Force
}

# Instead of pointing to normal efisys.bin, use the *_noprompt instead
if($XML.Configuration.ImageData.BootPrompt -eq "true"){
    $BootFile = "efisys.bin"
} Else {
    $BootFile = "efisys_noprompt.bin"
}

$BootData='2#p0,e,b"{0}"#pEF,e,b"{1}"' -f
"$($XML.Configuration.ImageData.StagingFolder)\boot\etfsboot.com", "$($XML.Configura
tion.ImageData.StagingFolder)\efi\Microsoft\boot\$BootFile"

# Create the ISO File
Start-Process -FilePath $($XML.Configuration.ImageData.Oscding) -
ArgumentList @("-bootdata:$BootData",'-u2','-'
udfver102'," $($XML.Configuration.ImageData.StagingFolder)","""$ISOFile""") -
PassThru -Wait -NoNewWindow | Out-Null

# Clean the Staging folder
If(Test-Path $XML.Configuration.ImageData.StagingFolder){
    Remove-Item -Path $XML.Configuration.ImageData.StagingFolder -Recurse -
Force
}

# Remove the MDT Source WIM File
if($XML.Configuration.ImageData.RemoveSourceWim -eq "true"){
    Remove-Item $WIMFile.Fullname -Force
} Else {
    Rename-Item -Path $WIMFile.Fullname -NewName

```

```

$($WIMFile.Fullname.Substring(0,$($WIMFile.FullName).Length-3) + "processed") -
Force
    }

    Write-EventLog -EventId 11 -LogName Application -Message "Succesfully
created $ISOFile" -Source SecureDeployment -Category 0 -EntryType Information
    }
} Else {
    Write-EventLog -EventId 12 -LogName Application -Message "No WIM Files could be
located." -Source SecureDeployment -Category 0 -EntryType Information
    Write-Output "No WIM Files could be located."
    Return
}

```

XMLConfigFile.xml

```

<?xml version="1.0" encoding="utf-8"?>
<Configuration>
    <OperatingSystems>
        <OperatingSystem name="OSD001 - Windows Server 2012 R2 Standard Desktop
Experience" key="OSD001CDrive">
            <Source>D:\DeploymentShare\Operating Systems\Windows Server 2012
R2</Source>
            <ImageName>Windows Server 2012 R2 Standard Evaluation (Server with a
GUI)</ImageName>
        </OperatingSystem>
        <OperatingSystem name="OSD005 - Windows Server 2022 Standard Evaluation
(Desktop Experience)" key="OSD005CDrive">
            <Source>D:\DeploymentShare\Operating Systems\Windows Server
2022</Source>
            <ImageName>Windows Server 2022 Standard Evaluation (Desktop Experience)
</ImageName>
        </OperatingSystem>
        <OperatingSystem name="OSD006 - Windows Server 2022 Standard Core
Evaluation" key="OSD006CDrive">
            <Source>D:\DeploymentShare\Operating Systems\Windows Server
2022</Source>
            <ImageName>Windows Server 2022 Standard Evaluation</ImageName>
        </OperatingSystem>
        <OperatingSystem name="OSD009 - Windows Server 2016 Standard Evaluation
(Desktop Experience)" key="OSD009CDrive">
            <Source>D:\DeploymentShare\Operating Systems\Windows Server
2016</Source>
            <ImageName>Windows Server 2016 Standard Evaluation (Desktop Experience)
</ImageName>
        </OperatingSystem>
        <OperatingSystem name="OSD013 - Windows Server 2019 Standard Evaluation
(Desktop Experience)" key="OSD013CDrive">
            <Source>D:\DeploymentShare\Operating Systems\Windows Server
2019</Source>
            <ImageName>Windows Server 2019 Standard Evaluation (Desktop Experience)
</ImageName>
        </OperatingSystem>
    </OperatingSystems>

```

```

        <OperatingSystem name="OSD014 - Windows Server 2019 Standard Core
Evaluation" key="OSD014CDrive">
            <Source>D:\DeploymentShare\Operating Systems\Windows Server
2019</Source>
            <ImageName>Windows Server 2019 Standard Evaluation</ImageName>
        </OperatingSystem>
        <OperatingSystem name="OSD017 - Windows 11 Enterprise Evaluation Build
22H2" key="OSD017CDrive">
            <Source>D:\DeploymentShare\Operating Systems\Windows 11 Enterprise -
22H2</Source>
            <ImageName>Windows 11 Enterprise Evaluation</ImageName>
        </OperatingSystem>
        <OperatingSystem name="OSD025 - Windows 10 Enterprise Evaluation Build
22H2" key="OSD025CDrive">
            <Source>D:\DeploymentShare\Operating Systems\Windows 10 Enterprise -
22H2</Source>
            <ImageName>Windows 10 Enterprise Evaluation</ImageName>
        </OperatingSystem>
    </OperatingSystems>
    <ImageData>
        <CaptureFolder>D:\DeploymentShare\Captures</CaptureFolder>
        <StagingFolder>D:\Staging</StagingFolder>
        <DestinationFolder>D:\Share</DestinationFolder>
        <Oscdimg>C:\Program Files (x86)\Windows Kits\10\Assessment and Deployment
Kit\Deployment Tools\amd64\Oscdimg\oscdimg.exe</Oscdimg>
        <DISM>C:\Program Files (x86)\Windows Kits\10\Assessment and Deployment
Kit\Deployment Tools\amd64\DISM\dism.exe</DISM>
        <ImageX>C:\Program Files (x86)\Windows Kits\10\Assessment and
Deployment Kit\Deployment Tools\amd64\DISM\imagex.exe</ImageX>
        <BootPrompt>True</BootPrompt>
        <RemoveSourceWim>False</RemoveSourceWim>
        <Compression>Max</Compression>
    </ImageData>
</Configuration>

```

New-WindowsInstallationMedia

XMLFile: The full path to the configuration file.

XMLConfigFile: Understanding the XML configuration options

- *OperatingSystem Name*: The name of the image we need to extract from the newly generated WIM file. We need this name as a reference.
- *OperatingSystem Key*: Build from MDT Task Sequence ID, Captured Drive Letter, The name "Drive".
- *OperatingSystem/Source*: Directory containing the original media files.
- *OperatingSystem/ImageName*: The name of the ISO file. Note, a creation date will be added.
- *ImageData/CaptureFolder*: The directory where the MDT captured images are stored.

- *ImageData/StagingFolder*: A temporary folder used during staging. This folder is created and removed during the process.
- *ImageData/DestinationFolder*: The directory path where the newly generated ISO files are stored.
- *ImageData/Oscdimg*: Full path to the OSCDIMG tool.
- *ImageData/DISM*: Full path to the DISM tool.
- *ImageData/ImageX*: Full path to the ImageX tool.
- *ImageData/BootPrompt*: Add or remove the 'Press any Key' before boot.
- *ImageData/RemoveSourceWim*: Remove the captured MDT WIM file after processing.
- *ImageData/Compression*: Compression type for the new WIM files. Valid options are, "None", "Fast", "Max" (default) or "Recovery".

Note! Although the compression option "Recovery" provides the smallest WIM file, generating the file will take a long time (> 25 minutes). Using "Max", will greatly speed up the generation of the ISO file and produce an acceptable size ISO file.

Setting up the scheduled task

With our PowerShell script and XML configuration file ready, the next step in our process is to create the scheduled task that will use these tools to automate the generation of the ISO files.

While it's possible to create a scheduled task manually, we want to maintain the predictability and consistency of our process. To this end, we will use PowerShell to create the scheduled task. This approach ensures that the task is created accurately and can be replicated exactly as needed, aligning with our 'clean source' principle.

Our task will be configured to run every 4 hours. At each run, it will scan the capture folder for new WIM images. If it finds any, it will process them into an updated ISO file based on the settings defined in our XML configuration file.

Here's how you can use PowerShell to create this scheduled task:

New-WindowsInstallationSchedule.ps1

```
$TaskName = "New-WindowsInstallationMedia"
$TaskDescription = "Create a new ISO file from an updated wim file"
$TaskFilePath = "D:\Create Installation Media\New-WindowsInstallationMedia.ps1"
$TaskInputXMLFile = "D:\Create Installation Media\XMLConfigFile.xml"
```

```
If (!(Get-ScheduledTask -TaskName $TaskName -ErrorAction SilentlyContinue)){
```

```
    $TaskAction = New-ScheduledTaskAction -Execute 'Powershell.exe' `
        -Argument ('-NoProfile -WindowStyle Hidden -ExecutionPolicy Bypass -file "' +
($TaskFilePath) + '" -XMLFile "' + ($TaskInputXMLFile) + '"')
```

```

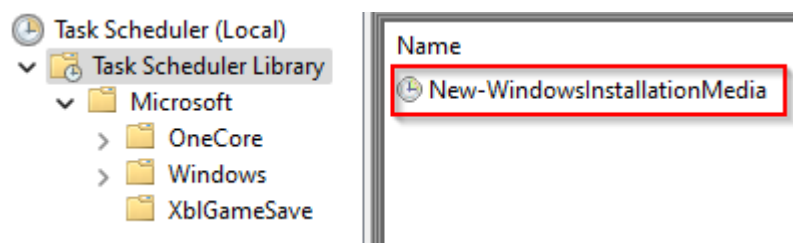
$TaskTrigger = New-ScheduledTaskTrigger -Once `
    -at (Get-Date).AddHours(1) `
    -RepetitionInterval (New-TimeSpan -Hours 4)

$TaskSettings = New-ScheduledTaskSettingsSet -ExecutionTimeLimit (New-TimeSpan
-Hours 1) `
    -RestartCount 3 `
    -RestartInterval (New-TimeSpan -Hours 1) `
    -DontStopIfGoingOnBatteries:$true `
    -AllowStartIfOnBatteries:$true

Register-ScheduledTask -Action $TaskAction `
    -Trigger $TaskTrigger `
    -TaskName $TaskName `
    -Description $TaskDescription `
    -Settings $TaskSettings `
    -User "System" -RunLevel Highest
}

```

PowerShell

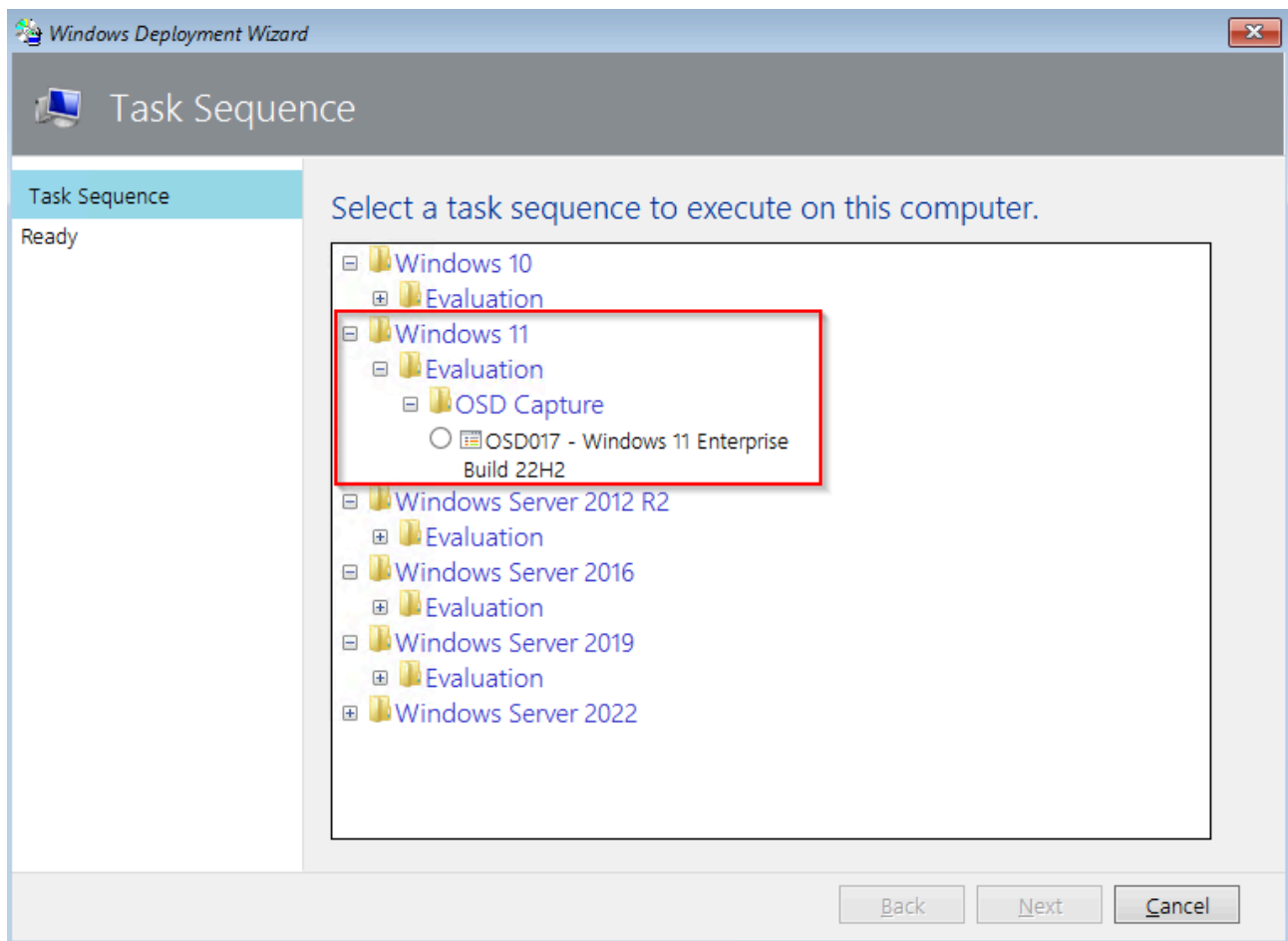


By automating the creation of the scheduled task, we further ensure the reliability and predictability of our process, bringing us another step closer to our goal of creating a secure, automated environment for generating up-to-date ISO files.

Putting it all into practice

We have now set the stage with our scripts, scheduled tasks, and configuration files. It's time to see everything in action and watch our automated environment for generating up-to-date ISO files work its magic.

First, we will start our virtual machine if it's not automatically started. Upon boot, it will run from the LiteTouch boot media and the MDT Deployment Wizard will start.



In the Deployment Wizard, we will select the operating system we want to install. The virtual machine will then proceed with the installation, including any new updates available at the time. Once the operating system is installed and updated, a new WIM image will be created.

With our new WIM image available, we simply need to wait for the scheduled task to kick in. Remember, our task runs every 4 hours and will automatically process any new WIM images into an updated ISO file, following the settings defined in our XML configuration file.

However, if you're like me and love to see results, you don't have to wait for the scheduled task to run automatically. You can manually trigger the task at any time to process the new WIM image.

This process encapsulates the 'clean source' principle in action: a predictable and verifiable way to create secure, up-to-date ISO files. By implementing these steps, we ensure the reliability of our software and strengthen the cybersecurity of our environment.

The final touch: Automating the deployment

Just when you thought we were done, there's one more thing I'd like to share with you that can further streamline our process: automating the MDT Deployment Wizard. With a slight modification to the CustomSettings.ini file, we can skip the Wizard entirely, automatically

select the task sequence, and kickstart the installation process. We achieve this automation by using the MAC address of our installation virtual machine.

To automate the Deployment Wizard, we will first need to retrieve the MAC address of our installation VM. Thankfully, PowerShell comes to the rescue once again, providing a quick and easy way to fetch the MAC address. Here's how:

PowerShell

```
Get-VM | Get-VMNetworkAdapter | ft VMName, MacAddress
```

With the MAC address in hand, we now proceed to alter the `CustomSettings.ini` file. This file, located in the Control folder of your deployment share, governs many aspects of the MDT deployment process. By adding a few lines tied to the MAC address of our VM, we can automate the Deployment Wizard.

INI

```
[Settings]
Priority=MACAddress, Default
Properties=MyCustomProperty
```

```
[Default]
OSInstall=Y
_SMSTSORGNAME=LAB
SkipAdminPassword=YES
SkipApplications=YES
SkipBitLocker=YES
SkipCapture=YES
SkipComputerBackup=YES
SkipDomainMembership=YES
SkipLocaleSelection=YES
SkipProductKey=YES
SkipUserData=YES
SkipTimeZone=YES
SkipComputerName=YES
SkipFinalSummary=YES
```

```
UILanguage=en-us
UserLocale=nl-nl
Systemlocale=en-US
KeyboardLocale= 0413:00000409
TimeZoneName=W. Europe Standard Time
```

```
WUMU_ExcludeKB001=4023307
WUMU_ExcludeKB002=4481252
```

```
[00:15:5D:15:D8:05]
TaskSequenceID=0SD017
SkipTaskSequence=YES
SkipSummary=YES
```

In the example I've added "*MACAddress*" in the [settings] section as a higher priority than the default settings, overruling the settings used during the default deployment. Next, I've added the MAC address of the VM between brackets at the end of the file and assigned a specific TaskSequenceID, instructed the deployment wizard to skip the Task Sequence question and the final summary.

With this final touch, our automated environment for generating up-to-date ISO files is now truly hands-off. The process is initiated, runs its course, and completes without any need for manual intervention, embodying the 'clean source' principle to its fullest.

Conclusion

By leveraging the power of all these tools, we have built an environment that consistently creates ISO files from a clean source. This practice aligns with the clean source principle, a cornerstone of cybersecurity, and ensures the authenticity and security of our deployed software.

Keeping cybersecurity principles in mind is crucial in a world increasingly reliant on digital solutions. As we've demonstrated here, tools like MDT and PowerShell can be powerful allies in maintaining a secure environment.

Download all the supporting files from this blog [here](#).